

Reviewer Pack — Minimal Rank of Algebraic Holography Bounds Communication Co...

Entry 5291bd084d1c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 14:12:03 UTC

Minimal Rank of Algebraic Holography Bounds Communication Complexity

Entry ID: 5291bd084d1c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 14:12:03 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Holographic Entanglement) **Field B** (complexity object): Communication Complexity (Entanglement Distillation)

Statement:

{‘quantitative’: ‘For all instances of entanglement distillation with n qubits, the minimal rank of the algebraic hologram of its maximally entangled state is $\Theta(\log n)$.’, ‘falsifier’: ‘There exists an instance of entanglement distillation with n qubits for which the minimal rank of the algebraic hologram is $o(\log n)$.’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘Algebraic topology, particularly algebraic holography, provides a framework to encode quantum states into algebraic structures. This conjecture suggests that the complexity of entanglement distillation can be understood through the rank of these algebraic encodings.’, ‘potential structure’: ‘This bridge may expose the underlying geometric and topological structures in the process of entanglement distillation, which could lead to new algorithms or insights into quantum information theory.’}

Taxonomy category: AlgebraicHolographyEntanglement (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 47e9508146ceaf35

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman’s rank correlation coefficient between the minimal rank of algebraic holograms and $\log n$ across 30 seeds is ≥ 0.8 , AND no seed shows a minimal rank less than half of $\log n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic topology" AND "holographic entanglement" AND communication complexity - "entanglement distillation" AND "communication complexity" AND "minimal rank" AND algebraic hologram - "maximally entangled state" AND "minimal rank" AND algebraic holography AND $\Theta(\log n)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_entangled_state(n):
        return [1 / math.sqrt(2)] * n + [-1 / math.sqrt(2)] * n

    def algebraic_hologram(state):
        n = len(state)
        hologram = []
        for i in range(n):
            for j in range(i+1, n):
                hologram.append((i, j, state[i] * state[j]))
        return hologram

    def min_rank(hologram):
        if not hologram:
            return 0
        rank = 0
        rows = []
        for i, (r1, r2, val) in enumerate(hologram):
            row = [val]
            for j, (s1, s2, v) in enumerate(hologram):
                if i != j:
                    row.append(v * state[s1] * state[s2])
```

```

        rows.append(row)
    while rows:
        max_row_index = 0
        max_val = abs(rows[0][0])
        for i, row in enumerate(rows):
            if abs(row[0]) > max_val:
                max_row_index = i
                max_val = abs(row[0])
        pivot_row = rows.pop(max_row_index)
        rank += 1
        for i, row in enumerate(rows):
            factor = -row[0] / pivot_row[0]
            rows[i] = [pivot_row[j] + factor * row[j] for j in range(len(pivot_row))]
    return rank

n_values = [5, 10, 15, 20, 30, 40]
ranks = []

for n in n_values:
    state = generate_entangled_state(n)
    hologram = algebraic_hologram(state)
    rank = min_rank(hologram)
    ranks.append(rank)

metric_value = sum(ranks) / len(ranks)
conjecture_holds = all(rank >= 0.5 * math.log(n, 2) for n, rank in zip(n_values, ranks))
counterexample = "" if conjecture_holds else "minimal_rank_less_than_half_log_n"

return {
    "metric_name": "min_rank",
    "metric_value": metric_value,
    "instances_tested": len(ranks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=minimal_rank_less_than_half_log_n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d7caea5.py", line 83, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d7caea5.py", line 63, in run_trial
    rank = min_rank(hologram)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0d7caea5.py", line 53, in min_rank
    factor = -row[0] / pivot_row[0]
              ~~~~~^~~~~~
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error, preventing the evaluation of the conjecture's support conditions. | next: Investigate and fix the division by zero issue in the code to allow for proper testing of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11952
2	preregistration	ollama_remote	glm4:latest	0	0	5559
3	novelty	ollama_remote	glm4:latest	0	0	5003
4	novelty	ollama_remote	glm4:latest	0	0	5302
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41104
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7826
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9983
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10351
9	judge	ollama_remote	glm4:latest	0	0	29011

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 126091 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5291bd084d1c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5291bd084d1c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5291bd084d1c.tar.gz` (if generated)